

Final Exam

Task 1

Convolutional Neural Network (CNN)

A Convolutional Neural Network (CNN) is a specialized deep learning architecture designed to automatically and adaptively learn spatial hierarchies of features from input data. CNNs are built using three core types of layers: convolutional layers, pooling layers, and fully connected layers. Convolutional layers apply learnable filters that slide across the input, detecting local patterns such as edges or textures. Pooling layers reduce spatial dimensions, decreasing computational load and controlling overfitting. Fully connected layers then interpret the extracted features to perform final classification. CNNs are highly effective for structured data with spatial relationships—most commonly images. However, CNNs are also useful in cybersecurity, especially for transforming network traffic into structured representations (e.g., matrices) that can reveal malicious patterns. For instance, converting packet-level features into a 2D tensor allows CNNs to detect anomalies such as DDoS attacks by learning characteristic traffic signatures.

Below is an example using a small synthetic dataset representing network traffic. The CNN attempts to classify samples as "benign" or "malicious."

src_bytes	dst_bytes	flag	label
120	450	1	benign
5000	23000	0	malicious
320	180	1	benign

CNN Model Architecture Definition

The following code defines the Convolutional Neural Network (CNN) architecture using the Keras Sequential API. The model consists of a 2D convolutional layer for feature extraction, a max-pooling layer for dimensionality reduction, a flattening layer to convert feature maps into a vector, and dense layers for classification.

```
import numpy as np
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Conv2D, Flatten, Dense, MaxPooling2D
# Example synthetic dataset (converted to 2D matrix form)
X = np.random.rand(100, 4, 4, 1) # 100 samples of 4x4 matrices
y = np.random.randint(0, 2, 100)
model = Sequential([
    Conv2D(8, (2,2), activation='relu', input_shape=(4,4,1)),
    MaxPooling2D((2,2)),
    Flatten(),
    Dense(16, activation='relu'),
    Dense(1, activation='sigmoid')
])
model.compile(optimizer='adam', loss='binary_crossentropy', metrics=['accuracy'])
model.fit(X, y, epochs=5, batch_size=8)
```